

Design and Analysis of Algorithms

Divide and Conquer III

Can we multiply without multiplying?

Fourier says yes — and then, a paradigm that never looks back.

Dr. Golzari

University of Hormozgan

Polynomial Multiplication

Question: How Fast Is Naive Multiplication?

$$A(x) = \sum a_i x^i, \quad B(x) = \sum b_i x^i \quad \Rightarrow \quad c_k = \sum_{i+j=k} a_i b_j$$

Question: How Fast Is Naive Multiplication?

$$A(x) = \sum a_i x^i, \quad B(x) = \sum b_i x^i \quad \Rightarrow \quad c_k = \sum_{i+j=k} a_i b_j$$

- Each c_k sums $\Theta(n)$ products; there are $\Theta(n)$ of them: $\Theta(n^2)$.

Question: How Fast Is Naive Multiplication?

$$A(x) = \sum a_i x^i, \quad B(x) = \sum b_i x^i \quad \Rightarrow \quad c_k = \sum_{i+j=k} a_i b_j$$

- Each c_k sums $\Theta(n)$ products; there are $\Theta(n)$ of them: $\Theta(n^2)$.
- **Question:** can we avoid touching every pair (a_i, b_j) ?

Example: Multiply $1 + 2x$ by $3 + 4x$

$$c_0 = 1 \cdot 3 = 3 \quad c_1 = 1 \cdot 4 + 2 \cdot 3 = 10 \quad c_2 = 2 \cdot 4 = 8$$

Example: Multiply $1 + 2x$ by $3 + 4x$

$$c_0 = 1 \cdot 3 = 3 \quad c_1 = 1 \cdot 4 + 2 \cdot 3 = 10 \quad c_2 = 2 \cdot 4 = 8$$

$$(1 + 2x)(3 + 4x) = 3 + 10x + 8x^2$$

Example: Multiply $1 + 2x$ by $3 + 4x$

$$c_0 = 1 \cdot 3 = 3 \quad c_1 = 1 \cdot 4 + 2 \cdot 3 = 10 \quad c_2 = 2 \cdot 4 = 8$$

$$(1 + 2x)(3 + 4x) = 3 + 10x + 8x^2$$

- Coefficient by coefficient — the **convolution** of the two lists.

Key Insight: Polynomials Have Two Faces

Coefficient form

$a_0, a_1, \dots, a_{n-1}$ Multiplication =
convolution: $\Theta(n^2)$

Value form

$\{(x_1, A(x_1)), \dots\}$ Multiplication =
pointwise: $\Theta(n)$

Key Insight: Polynomials Have Two Faces

Coefficient form

$a_0, a_1, \dots, a_{n-1}$ Multiplication =
convolution: $\Theta(n^2)$

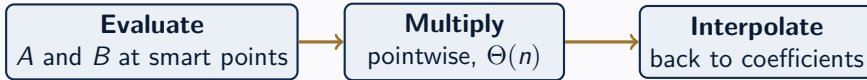
Value form

$\{(x_1, A(x_1)), \dots\}$ Multiplication =
pointwise: $\Theta(n)$

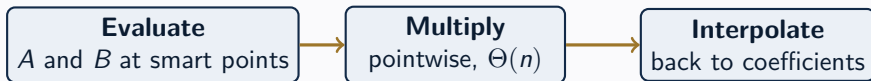
The catch

The product has degree $2n - 2$, so $2n - 1$ point pairs are needed — and converting between the two forms costs work.

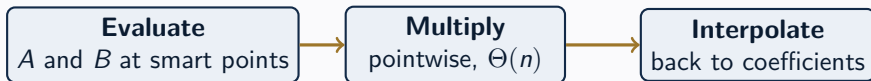
The Plan



The Plan



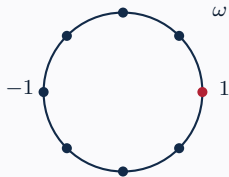
- Naive evaluation: $\Theta(n)$ per point, $2n - 1$ points $\Rightarrow \Theta(n^2)$. Same as before!



- Naive evaluation: $\Theta(n)$ per point, $2n - 1$ points $\Rightarrow \Theta(n^2)$. Same as before!
- **Question:** can we choose the points so that evaluation is nearly free?

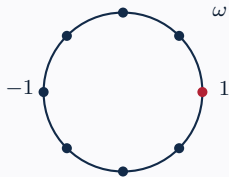
The Fast Fourier Transform

The Magic Points: Roots of Unity



Points: $1, \omega, \omega^2, \dots, \omega^{n-1}, \quad \omega = e^{2\pi i/n}.$

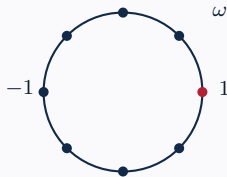
The Magic Points: Roots of Unity



Points: $1, \omega, \omega^2, \dots, \omega^{n-1}, \quad \omega = e^{2\pi i/n}.$

- Squaring maps them onto the $n/2$ -th roots — **each point twice**: $\omega^{j+n/2} = -\omega^j$, same square.

The Magic Points: Roots of Unity



Points: $1, \omega, \omega^2, \dots, \omega^{n-1}, \quad \omega = e^{2\pi i/n}.$

- Squaring maps them onto the $n/2$ -th roots — **each point twice**: $\omega^{j+n/2} = -\omega^j$, same square.
- Same family, half size — **recursion loves this**.

Divide: Even and Odd Coefficients

$$A(x) = \underbrace{A_e(x^2)}_{\text{even coeffs}} + x \cdot \underbrace{A_o(x^2)}_{\text{odd coeffs}}$$

Divide: Even and Odd Coefficients

$$A(x) = \underbrace{A_e(x^2)}_{\text{even coeffs}} + x \cdot \underbrace{A_o(x^2)}_{\text{odd coeffs}}$$

For each j (with $y = (\omega^j)^2$):

$$A(\omega^j) = A_e(y) + \omega^j A_o(y) \quad A(-\omega^j) = A_e(y) - \omega^j A_o(y)$$

Divide: Even and Odd Coefficients

$$A(x) = \underbrace{A_e(x^2)}_{\text{even coeffs}} + x \cdot \underbrace{A_o(x^2)}_{\text{odd coeffs}}$$

For each j (with $y = (\omega^j)^2$):

$$A(\omega^j) = A_e(y) + \omega^j A_o(y) \quad A(-\omega^j) = A_e(y) - \omega^j A_o(y)$$

- Two half-size FFTs (on A_e , A_o) give **two** full-size values.
- $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ (same shape as merge sort).

Worked Example: FFT of Size 4

$$A(x) = 1 + 2x + 3x^2 + 4x^3, \quad \omega = i \quad (\text{points } 1, i, -1, -i).$$

Worked Example: FFT of Size 4

$$\begin{aligned} A(x) &= 1 + 2x + 3x^2 + 4x^3, & \omega &= i & (\text{points } 1, i, -1, -i). \\ A_e(y) &= 1 + 3y, & A_o(y) &= 2 + 4y, & \text{half-points } y \in \{1, -1\}. \end{aligned}$$

Worked Example: FFT of Size 4

$$A(x) = 1 + 2x + 3x^2 + 4x^3, \quad \omega = i \quad (\text{points } 1, i, -1, -i).$$

$$A_e(y) = 1 + 3y, \quad A_o(y) = 2 + 4y, \quad \text{half-points } y \in \{1, -1\}.$$

x	$y = x^2$	$A_e(y), A_o(y)$	$A(x)$
1	1	4, 6	$4 + 6 = 10$
i	-1	-2, -2	$-2 - 2i$
-1	1	4, 6	$4 - 6 = -2$
$-i$	-1	-2, -2	$-2 + 2i$

Worked Example: FFT of Size 4

$$A(x) = 1 + 2x + 3x^2 + 4x^3, \quad \omega = i \quad (\text{points } 1, i, -1, -i).$$

$$A_e(y) = 1 + 3y, \quad A_o(y) = 2 + 4y, \quad \text{half-points } y \in \{1, -1\}.$$

x	$y = x^2$	$A_e(y), A_o(y)$	$A(x)$
1	1	4, 6	$4 + 6 = 10$
i	-1	-2, -2	$-2 - 2i$
-1	1	4, 6	$4 - 6 = -2$
$-i$	-1	-2, -2	$-2 + 2i$

Check directly: $A(i) = 1 + 2i - 3 - 4i = -2 - 2i$. ✓

Interpolation Is Just Another FFT

$$c_k = \frac{1}{n} \sum_j C(\omega^j) \omega^{-kj}$$

Interpolation Is Just Another FFT

$$c_k = \frac{1}{n} \sum_j C(\omega^j) \omega^{-kj}$$

- Same halving trick, with ω^{-1} instead of ω : $\Theta(n \log n)$.

Interpolation Is Just Another FFT

$$c_k = \frac{1}{n} \sum_j C(\omega^j) \omega^{-kj}$$

- Same halving trick, with ω^{-1} instead of ω : $\Theta(n \log n)$.
- Pipeline: two FFTs + pointwise + inverse FFT

$$\Theta(n \log n) + \Theta(n) + \Theta(n \log n) = \Theta(n \log n)$$

What This Buys

Method	Cost
Naive convolution	n^2
Karatsuba	$n^{1.585}$
FFT	$n \log n$

What This Buys

Method	Cost
Naive convolution	n^2
Karatsuba	$n^{1.585}$
FFT	$n \log n$

- Digits are coefficients: FFT multiplies **million-digit integers**.

What This Buys

Method	Cost
Naive convolution	n^2
Karatsuba	$n^{1.585}$
FFT	$n \log n$

- Digits are coefficients: FFT multiplies **million-digit integers**.
- JPEG / MP3: transform, keep the biggest coefficients, drop the rest.

Introduction to Greedy

Question: Can We Decide Without Recursing?

Divide & conquer

- Split, solve **all** pieces.
- Combine the answers.
- Recursion everywhere.

Greedy

- One decision per step.
- Each decision shrinks the problem.
- No combine, no second branch.

Question: Can We Decide Without Recursing?

Divide & conquer

- Split, solve **all** pieces.
- Combine the answers.
- Recursion everywhere.

Greedy

- One decision per step.
- Each decision shrinks the problem.
- No combine, no second branch.

The risk

No plan B. Whatever we choose, we live with it — the algorithm never reconsiders.

The Greedy Mindset



The Greedy Mindset



$\underbrace{\text{locally best}}_{\text{each step}} \stackrel{?}{\Rightarrow} \underbrace{\text{globally best}}_{\text{final answer}}$

The Greedy Mindset



$$\underbrace{\text{locally best}}_{\text{each step}} \stackrel{?}{\Rightarrow} \underbrace{\text{globally best}}_{\text{final answer}}$$

- The ? is the designer's job: local best is a claim, not a proof.

Greedy in Action

Warm-Up: Coin Change — It Works

Coins $\{25, 10, 5, 1\}$, amount 63. Greedy rule: **take the largest coin** $\leq$ remainder.

Warm-Up: Coin Change — It Works

Coins $\{25, 10, 5, 1\}$, amount 63. Greedy rule: **take the largest coin** $\leq$ remainder.

$$63 \xrightarrow{25} 38 \xrightarrow{25} 13 \xrightarrow{10} 3 \xrightarrow{1} 2 \xrightarrow{1} 1 \xrightarrow{1} 0$$

Warm-Up: Coin Change — It Works

Coins $\{25, 10, 5, 1\}$, amount 63. Greedy rule: **take the largest coin** $\leq$ remainder.

$$63 \xrightarrow{25} 38 \xrightarrow{25} 13 \xrightarrow{10} 3 \xrightarrow{1} 2 \xrightarrow{1} 1 \xrightarrow{1} 0$$

- 6 coins — and indeed optimal.

Warm-Up: Coin Change — It Works

Coins $\{25, 10, 5, 1\}$, amount 63. Greedy rule: **take the largest coin** $\leq$ remainder.

$$63 \xrightarrow{25} 38 \xrightarrow{25} 13 \xrightarrow{10} 3 \xrightarrow{1} 2 \xrightarrow{1} 1 \xrightarrow{1} 0$$

- 6 coins — and indeed optimal.
- So greedy always works for coin change, right?

Warning: The Same Greedy Can Fail

Coins $\{1, 3, 4\}$, amount 6.

$$6 \xrightarrow{4} 2 \xrightarrow{1} 1 \xrightarrow{1} 0 \quad 3 \text{ coins} \quad \text{vs.} \quad 3 + 3 = 6 \quad 2 \text{ coins}$$

Warning: The Same Greedy Can Fail

Coins $\{1, 3, 4\}$, amount 6.

$$6 \xrightarrow{4} 2 \xrightarrow{1} 1 \xrightarrow{1} 0 \quad 3 \text{ coins} \quad \text{vs.} \quad 3 + 3 = 6 \quad 2 \text{ coins}$$

Moral

Greedy is not an algorithm — it is a **claim**. Every greedy rule must come with a proof.

Proving Greedy: The Exchange Argument

1. Take any optimal solution **OPT**.

Proving Greedy: The Exchange Argument

1. Take any optimal solution **OPT**.
2. Show its first choice can be **exchanged** with the greedy choice, without making OPT worse.

Proving Greedy: The Exchange Argument

1. Take any optimal solution **OPT**.
 2. Show its first choice can be **exchanged** with the greedy choice, without making OPT worse.
 3. Repeat on the remainder $\Rightarrow$ greedy is as good as OPT.
- Needed: **greedy choice property** (a local best belongs to some optimal solution) + **optimal substructure** (the remainder solves independently).

Classic: Activity Selection

One room, n activities with start and finish times; pick the most non-overlapping ones.

Classic: Activity Selection

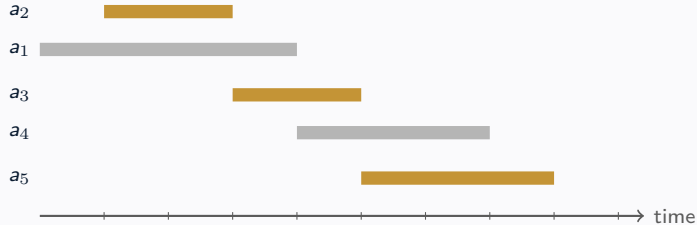
One room, n activities with start and finish times; pick the most non-overlapping ones.

Question: sort by what? Longest? Earliest start? Earliest *finish*?

Classic: Activity Selection

One room, n activities with start and finish times; pick the most non-overlapping ones.

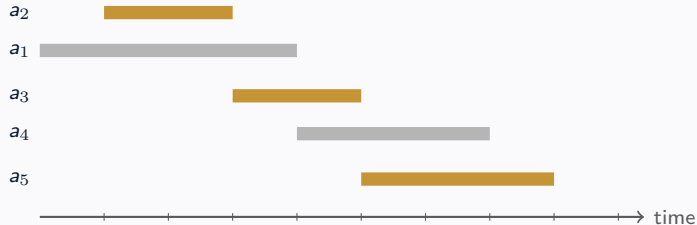
Question: sort by what? Longest? Earliest start? Earliest *finish*?



Classic: Activity Selection

One room, n activities with start and finish times; pick the most non-overlapping ones.

Question: sort by what? Longest? Earliest start? Earliest *finish*?



Sorted by finish: $a_2, a_1, a_3, a_4, a_5 \Rightarrow$ pick a_2, a_3, a_5 : **3 activities.**

Why Earliest Finish?

Finishing earliest leaves the **most room** for everything that follows — the greedy choice can always be exchanged into an optimal solution.

Why Earliest Finish?

Finishing earliest leaves the **most room** for everything that follows — the greedy choice can always be exchanged into an optimal solution.

- Exchange: OPT's first activity finishes *no earlier* than greedy's + it cannot overlap the rest of OPT after the swap.

Why Earliest Finish?

Finishing earliest leaves the **most room** for everything that follows — the greedy choice can always be exchanged into an optimal solution.

- Exchange: OPT's first activity finishes *no earlier* than greedy's + it cannot overlap the rest of OPT after the swap.
- Sorting by start or by length fails — **exercise**: build the counterexamples.

Wrap-Up

Key Takeaways

- Two faces of a polynomial: multiply pointwise in **value form** — $\Theta(n)$ instead of $\Theta(n^2)$.

Key Takeaways

- Two faces of a polynomial: multiply pointwise in **value form** — $\Theta(n)$ instead of $\Theta(n^2)$.
- Roots of unity survive squaring $\Rightarrow T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Key Takeaways

- Two faces of a polynomial: multiply pointwise in **value form** — $\Theta(n)$ instead of $\Theta(n^2)$.
- Roots of unity survive squaring $\Rightarrow T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.
- Greedy: **look – take – commit**; worth exactly as much as its exchange proof.

Key Takeaways

- Two faces of a polynomial: multiply pointwise in **value form** — $\Theta(n)$ instead of $\Theta(n^2)$.
- Roots of unity survive squaring $\Rightarrow T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.
- Greedy: **look** – **take** – **commit**; worth exactly as much as its exchange proof.
- Products: $n^2 \rightarrow n \log n$; decisions: one pass, no backtracking.

Next session

Greedy classics: Huffman coding, minimum spanning trees (Kruskal, Prim), Dijkstra.

1. Compute the size-4 FFT of $A(x) = 2 + x + x^2 + 3x^3$ by hand ($\omega = i$).
2. Why does multiplying two degree-1 polynomials need a size-4 FFT, not size 2? Compute the four pointwise products for $(1 + 2x)(3 + 4x)$.
3. Coins $\{1, 7, 10\}$, amount 14: show greedy fails and give the optimal solution.
4. Build interval sets where sorting by **length** and by **start time** both fail for activity selection.
5. **(Challenge)** Prove the earliest-finish greedy for activity selection with a full exchange argument.